

4.1 Directly-Mapped L1 Cache

We used the SimpleCache.c given by the professor as a starting point to our project. In our implementation, we used shifts and masks to get the tag, the index and the offset. We could have done the same thing using only divisions and mod and the macros from Cache.h (e.g.: $\text{index_n} = (\text{address} / \text{BLOCK_SIZE}) \% (\text{L1_SIZE} / \text{BLOCK_SIZE})$) but we believe it is easier to read our implementation even though it is not scalable. To calculate the number of lines of the cache we did $\text{L1_SIZE} / \text{BLOCK_SIZE}$.

4.2 Directly-Mapped L2 Cache

In this task, the both caches are similar to the L1 cache from the previous task. The only things that change are the tag bits for L2 and the fact that now, when there is a miss on the L1 cache, we first access the L2 cache and we only access the DRAM if we also miss on the L2 cache.

4.3 2-Way L2 Cache

This is the task where the code changes the most. We decided to use an array of arrays for the L2 cache (`L2Cache [ASSOCIATIVITY_L2] [L2_SIZE / ASSOCIATIVITY_L2]`) because we think that this is what best represents an actual L2 cache. Because of this, we also changed the L2Cache struct so that each line is now an array of arrays (`lines[(L2_SIZE / BLOCK_SIZE) / ASSOCIATIVITY_L2] [ASSOCIATIVITY_L2 ]`) where each line block has an int lru that is set to 1 if it is the least recently used block of the set.

The major difference we see in the code is in the accessL2 function. Now we have to check for a cache hit in both line blocks. If none of them have a hit we choose the LRU one. Every time we write to a L2 block we set the lru of that block to 0 and set the ix_block int to the index of the block that is going to be used when reading/writing data from/to the cache.

Tests

We also wrote 2 tests for our program: L1_test.c and All_test.c. The first one is a simple program that tests the L1 cache, with emphasis on the dirty bit. The second one is a stress test to see how the different caches handle misses and hits.

In L1_test.c we write to an address in L1 but because it is not valid, we have to read the value from DRAM (100 time units) and then write that value to the L1 cache (1 t.u.). Afterwards, we read from a different address but since it has the same index as the first one, it will go to the same cache line. However, the tag is not the same as the one that is already there, so it will read from DRAM (100 t.u.) and it will have to write back the value that was in the cache (50 t.u.) and only then it reads the value from the cache (1 t.u.). Finally, we read again from the same line with the address of the first access, so we will have a miss, read from the DRAM (100 t.u.) and read from the cache (1 t.u.). All the times above add up to 353 t.u. as per the test output.

In All_test.c we do a simple stress test where we do multiple reads. The following chart shows the access times

